

Sea Ice Classification

Using Deep Fusion of Satellite Imagery and Laser Photon Data

A Plain-Language Walkthrough of the Methodology

Santosh Pant
Research Seminar Project · May 2026

This document explains, in everyday language, what we are building, what data we use, how the model works, and how we measure success. No prior machine-learning experience required.

1. The Problem We Are Solving

The Arctic and Antarctic oceans are covered with sea ice that constantly forms, melts, drifts, and breaks apart. Knowing where the ice is, how thick it is, and how it is changing matters a lot - for climate science, for shipping routes through the Arctic, and for predicting how fast the polar regions are warming.

Our goal is to build a computer model that looks at a small image of a polar region and, for every pixel in that image, decides one of three things:

- **Ice** - solid, established sea ice (shown in red on our maps).
- **Thin ice** - newly forming or refreezing ice (shown in blue).
- **Water** - open ocean (shown in green).

This kind of pixel-by-pixel labeling is called **semantic segmentation**. The output is a colored map the same size as the input image.

2. The Data We Have

For each labeled point in our dataset, we have three pieces of information that all describe the same spot on Earth:

1. A satellite photo (Sentinel-2)

Sentinel-2 is a European satellite that takes color photos of Earth from space. We took the original large photos and cut out small 128 by 128 pixel squares centered on each labeled point. Think of it as a postage-stamp-sized aerial photo of a small piece of the Arctic Ocean.

2. A row of measurements from ICESat-2

ICESat-2 is a NASA satellite that fires a laser at the Earth's surface and counts the photons that bounce back. From these photon returns, scientists derive about 22 numbers for each measured point - things like the average height of the surface, how reflective it is, the spread of photon arrival times, and so on. These numbers come from a CSV file (basically a spreadsheet).

The key idea: ICESat-2 sees the surface in 1D - one measurement after another along the satellite's flight path. Sentinel-2 sees the surface in 2D - a full picture. Combining both gives the model more information than either alone.

3. A ground-truth segmentation map

Our professor's team built a hand-coded image-processing pipeline that converts each satellite photo into a colored map: every pixel is colored red, blue, or green for ice, thin ice, or water. This is the answer key our model learns to predict.

In total, we have **139,335** such (photo, CSV row, answer-key) triples, spread across **3 different geographic regions** (called "tiles") on **2 different dates** in November 2019.

3. The Goal, Stated Precisely

Given a new satellite photo and the matching CSV row, predict the colored answer-key map.

Inputs:

- 128 x 128 x 3 RGB photo (one piece of the Arctic seen from space).
- A small window of 32 nearby CSV rows from the ICESat-2 satellite track (so the model sees a bit of context, not just one isolated measurement).

Output:

- 128 x 128 prediction map, with each pixel labeled as ice, thin ice, or water.

4. How We Built the Training Data

Before any machine learning, we had to prepare the data. This was actually most of the work so far. The pipeline:

- Match each of the 6 CSV files to its matching satellite photo by tile code and date.
- For every labeled point in the CSV, look up its (column, row) pixel position inside the satellite photo and crop a 128 by 128 piece centered on that point.
- If the point is too close to the edge of the satellite photo, fill the missing area with black so every crop ends up the same size.
- Run the professor's image-processing pipeline (HSV color thresholding plus shadow and cloud removal) on each crop to produce the colored answer-key map.
- Save both the original photo and the answer-key map as PNG files, paired by name.

Result: two folders of 139,335 paired images each:

```
outputs/                <- original 128x128 RGB photos
outputs_segmented/      <- matching 128x128 answer-key maps
```

5. The Model - Deep Fusion

The model has two specialized parts (called "branches") that work in parallel, then a fusion step that combines what they learned.

Branch A: U-Net for the satellite photo

U-Net is a famous neural network architecture designed specifically for pixel-by-pixel image labeling. The intuition: imagine a person reading a map. First they zoom out to see the big picture - where are the major shapes, where are the boundaries. Then they zoom back in to label every fine detail. U-Net does exactly this, in two halves: an **encoder** that progressively shrinks the image to capture context, and a **decoder** that progressively re-expands to produce a full-resolution map.

We use a U-Net with a ResNet-18 backbone that has been pretrained on millions of everyday photos (ImageNet). That pretraining gives us a head-start: the network already knows about edges, textures, and shapes before it ever sees a single ice photo.

Branch B: Bi-LSTM for the CSV measurements

ICESat-2 produces measurements in a long line along its flight path. Adjacent measurements are about 10 meters apart on the ground, so they are physically related. An LSTM (Long Short-Term Memory network) is a neural network that reads sequences of numbers - like reading a sentence word by word and remembering the context as it goes.

For every labeled point, we hand the LSTM a window of 32 nearby rows (16 before, 16 after, ordered along the satellite track). The LSTM digests this sequence and outputs a single 256-number summary that captures the local along-track context. "Bi" in Bi-LSTM means it reads

the sequence in both directions, forwards and backwards, for richer context.

Fusion: combining the two branches

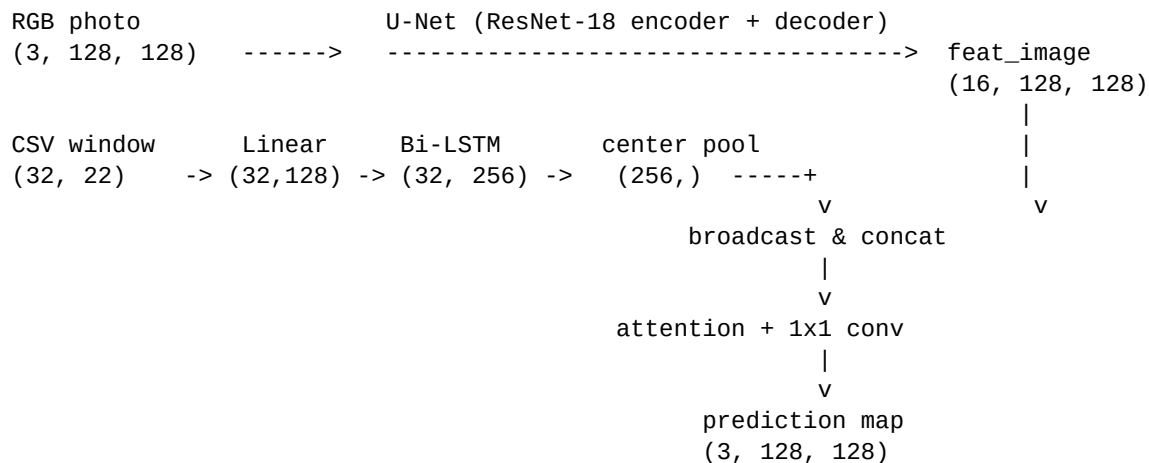
Now we have two outputs:

- From U-Net: a 16-channel feature map of size 128 x 128. Think of it as a 16-page atlas where each page highlights different aspects of the photo.
- From Bi-LSTM: a 256-number summary of the CSV neighborhood.

To combine them, we "broadcast" the 256-number CSV summary across the entire 128 x 128 spatial grid (so every pixel gets the same CSV summary attached to it), shrink it down to 16 channels, and concatenate with the image feature map. We now have 32 channels of fused information.

A small attention module (called Squeeze-and-Excitation) lets the model automatically decide which of the 32 channels are most useful. Finally, a 1 by 1 convolution turns the 32 channels into 3 channels - one prediction per class for every pixel.

Architecture diagram



Why "deep fusion"? Because we combine the two modalities at the feature level (deep inside the network) rather than at the input or at the output. The paper we are benchmarking against shows this works best.

6. How We Train the Model

Splitting the data

We split the 139,335 samples into three sets:

Split	Tiles	Samples	Purpose
Train	T02CNA + T02CNC	~84,000	model learns from these
Validation	10% held out from train	~9,300	tune choices, pick best epoch
Test	T03CWT (new tile, new date)	~46,000	final honest evaluation

We split **by region, not by row**. If we split randomly by row, neighboring rows (which have nearly identical photo crops) would end up in different sets, letting the model "cheat" by memorizing training examples. Splitting by region forces the model to generalize to genuinely new places.

Loss function

Our class distribution is heavily skewed: about 96% of pixels are ice, 3% thin ice, and 1% water. Without correction, the model would just say "ice" everywhere and be 96% accurate. To prevent this, we use **weighted cross-entropy loss**, which up-weights the rare classes so the model is heavily penalized for missing them.

Augmentation

During training, we randomly flip and rotate each photo (and its matching answer key). This teaches the model that ice looks like ice no matter the orientation - it cannot rely on "top of image is always sky-facing" because there is no sky here. This trick is called **data augmentation**.

Training loop details

Setting	Value	Why
Optimizer	AdamW	industry-standard, robust
Learning rate	1e-4 with cosine decay	starts moderate, ends fine-tuned
Batch size	32	fits comfortably on a single GPU
Epochs	30 (with early stopping)	stop if validation stops improving
Mixed precision	Yes	about 2x faster, same accuracy
Random seeds	3 (42, 7, 1337)	report mean and spread for reliability

7. How We Measure Success

Our primary metric is **mean Intersection over Union (mIoU)**. This is the standard metric for segmentation tasks - and it is what the paper we are benchmarking against uses.

For each class, we measure:

- Pixels we correctly predicted as that class (true positives).
- Pixels we predicted as that class but were not (false positives).
- Pixels that were that class but we missed (false negatives).

$\text{IoU} = \frac{\text{correctly_predicted}}{(\text{correctly_predicted} + \text{missed} + \text{over-predicted})}$. The score is between 0 and 1, where 1 is perfect.

mIoU is just the average IoU across all 3 classes. We also report:

- **Per-class IoU** - so we can see, for example, if the model is great at ice but bad at thin ice.
- **Pixel accuracy** - the simple percentage of pixels correctly labeled.
- **Confusion matrix** - a 3x3 grid showing exactly which classes the model confuses with which others.
- **Sample predictions** - 50 side-by-side comparisons of (input photo, true map, predicted map) so we can eyeball the results.

8. Ablation Study - Does Fusion Actually Help?

The whole point of this project is to show that combining the two modalities (photo + CSV) beats using either one alone. To prove this, we train **six versions** of the model and compare their mIoU:

#	Variant	Image branch	CSV branch	Fusion
1	Image only	U-Net	-	-
2	CSV only	-	Bi-LSTM	-
3	Early fusion	U-Net	-	concat at input
4	Late fusion	U-Net	Bi-LSTM	average predictions
5	Deep fusion (concat)	U-Net	Bi-LSTM	feature-level concat
6	Deep fusion (attention)	U-Net	Bi-LSTM	feature-level + attention

If everything works as expected, variants 5 and 6 (deep fusion) will outperform 1-4. This mirrors the comparison done in the benchmark paper and gives us a clean story: *combining modalities deep inside the network is the best way to fuse them*.

9. Comparison with the Benchmark Paper

Our work is directly inspired by:

Zhao, L., et al. (2023). "Deep-Learning-Based Sea Ice Classification With Sentinel-1 and AMSR-2 Data." IEEE Journal of Selected Topics in Applied Earth Observations and Remote Sensing, vol. 16, pp. 5514-5525.

Their setup is similar but not identical to ours:

Aspect	Their paper	Our project
Main image	Sentinel-1 SAR (radar, 2 channels)	Sentinel-2 (optical RGB)
Aux modality	AMSR-2 (microwave, 14-channel grid)	ICESat-2 (laser photons, CSV)
Aux shape	2D dense raster	1D sparse along-track sequence
Aux encoder	CNN	Bi-LSTM
Number of classes	12 (11 ice types + land)	3 (ice, thin ice, water)
Task	Per-pixel segmentation	Per-pixel segmentation
Primary metric	mIoU	mIoU

We are not trying to *beat* their numbers - the modalities and class counts are different so the numbers are not directly comparable. What we want to show is that **the same architectural pattern they discovered (deep feature-level fusion beats early and late fusion) also holds when the auxiliary modality is a sparse 1D track instead of a dense 2D grid**. That is the genuine contribution of this project.

10. What Comes Next

After we get the baseline numbers, possible extensions include:

- Adding more tiles and more dates to test how well the model generalizes across seasons.
- Replacing the LSTM with a Transformer for the CSV branch (Transformers often do better on long sequences).
- Adding Sentinel-1 SAR as a third modality, so we have radar + optical + photons.
- Trying self-supervised pretraining on millions of unlabeled satellite photos to learn better visual features before any fine-tuning.
- Producing high-resolution sea-ice maps for a full polar region by tiling and stitching predictions together.

Questions or feedback? This document is a living draft - corrections welcome.